

Assignment #02 Report

2D Cricket Web Application (Web Development)

Name	Masooma Ali
Roll Number	23i0623
Section	Section C
Course	CS-4032: Web Programming
Application	Cricket Blitz — 2D Cricket Batting Game
Technologies	React, HTML5 Canvas, CSS Modules, JavaScript

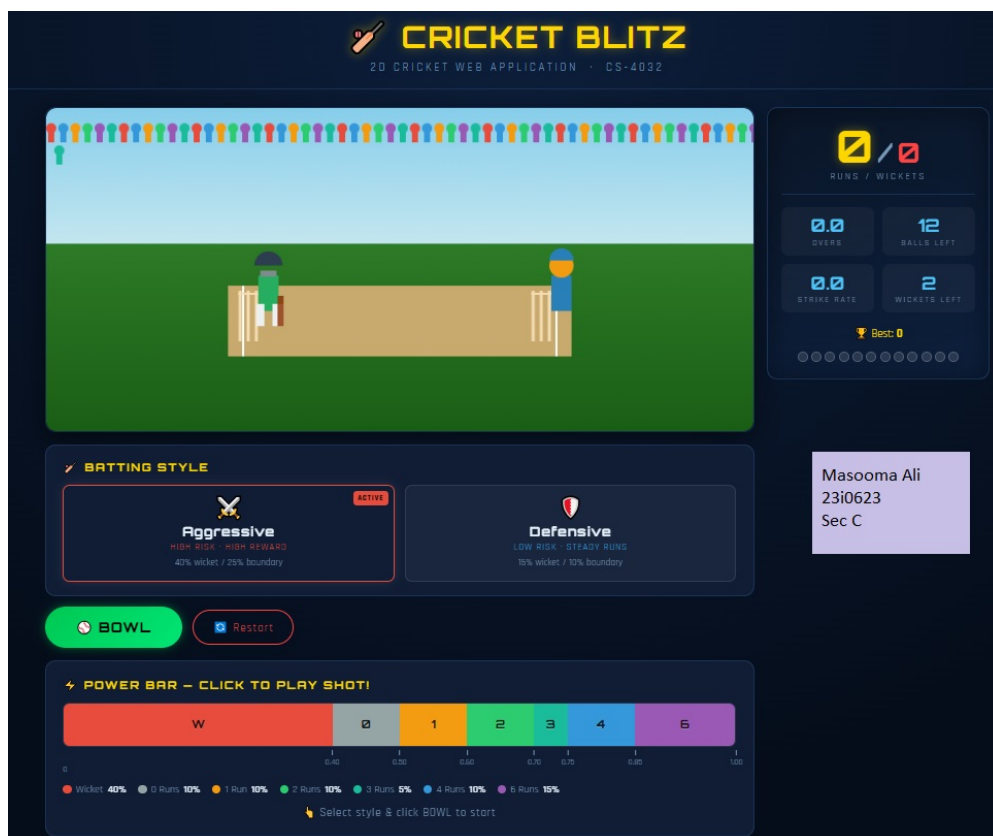


Figure 0 — Full gameplay overview of Cricket Blitz showing the cricket field, scoreboard, batting style selector, and power bar.

PART 1 — GAMEPLAY SCREENSHOTS

Each screenshot below captures a distinct stage of the game. A sticky note showing **Name**, **Roll Number**, and **Section** is visible in every screenshot as required.

SECTION 1.1

Aggressive Batting in Action

The Aggressive batting style is selected (shown by the ACTIVE badge). The power bar reflects the Aggressive probability distribution: the Wicket zone occupies 40% of the bar (the largest red segment), while the 6 Runs zone is 15% (purple). The scoreboard updates dynamically as each ball is bowled. The sticky note confirming student identity is visible on the right side of the screen.

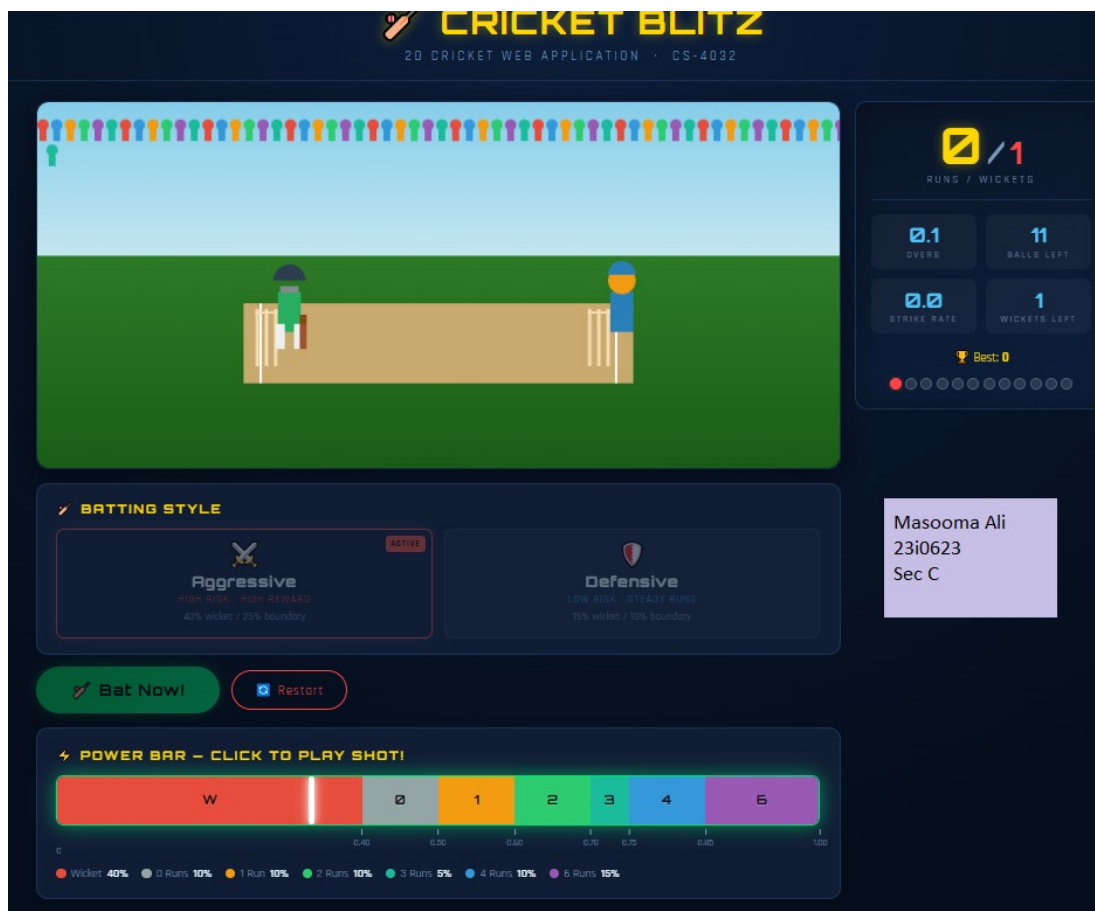


Figure 1.1 — Aggressive batting active. Power bar shows 40% Wicket zone. Score: 0/1, 0.1 overs, 11 balls left. Sticky note: Masooma Ali | 23i0623 | Sec C.

SECTION 1.2

Defensive Batting in Action

The Defensive batting style is now active. The power bar changes to reflect the Defensive distribution — the Wicket zone shrinks to only 15% (small red segment on the left), while the 0 Runs zone expands to 30% and the 1 Run zone takes 25%, reflecting a safer, run-accumulation strategy. The score has progressed to 11/0 after 0.5 overs with a strike rate of 220, demonstrating the defensive style building runs without losing wickets.



Figure 1.2 — Defensive batting active. Power bar shows 15% Wicket zone and 30% dot ball zone. Score: 11/0, 0.5 overs. Sticky note: Masooma Ali | 23i0623 | Sec C.

SECTION 1.3

Power Bar Visible During Gameplay

A close-up of the Power Bar showing the animated slider in mid-motion. The bar is divided into 7 colour-coded segments proportional to the Aggressive batting style probabilities. The white vertical slider travels continuously left to right across the bar. The probability scale (0 to 1.00) is displayed below the bar with tick marks at each segment boundary. The legend beneath shows each outcome with its exact percentage.

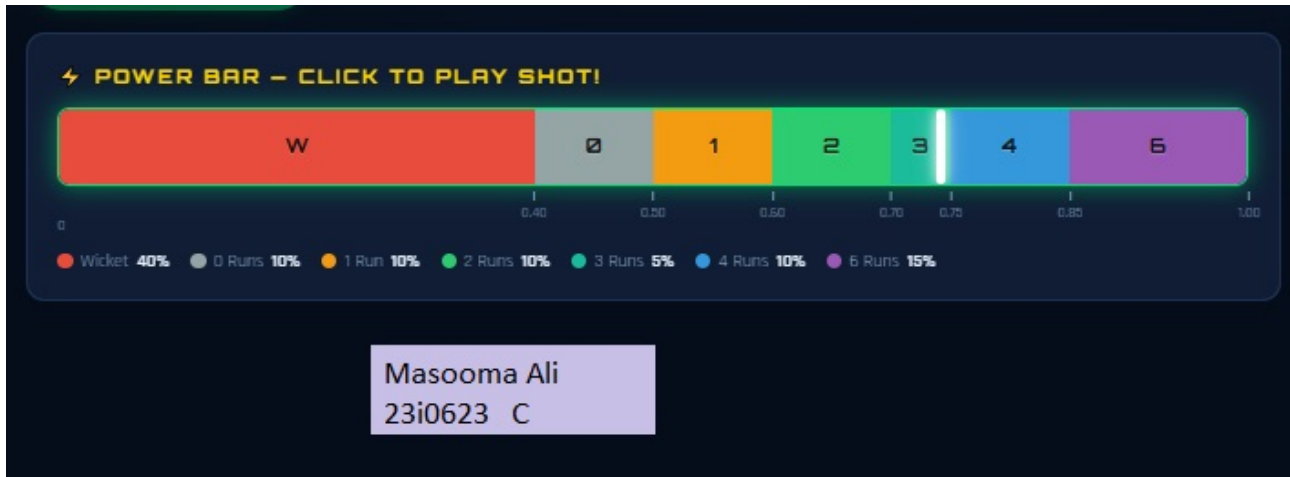


Figure 1.3 — Power bar during active gameplay. Slider visible mid-bar. Segments: W=40%, 0=10%, 1=10%, 2=10%, 3=5%, 4=10%, 5=15%. Sticky note: Masooma Ali | 23i0623 | Sec C.

SECTION 1.4

Scoreboard Updates Reflecting Game Progress

This screenshot captures the scoreboard mid-innings, showing real-time updates. The score reads 13/1 with 0.5 overs bowled and 7 balls remaining. The strike rate has risen to 260.0. The ball tracker at the bottom of the scoreboard shows 5 red dots (balls used) and 7 grey dots (balls remaining). The field shows the 'OUT!' overlay confirming a wicket was just taken, and the commentary box at the bottom reads 'Plumb LBW! That's stone dead! OUT!'

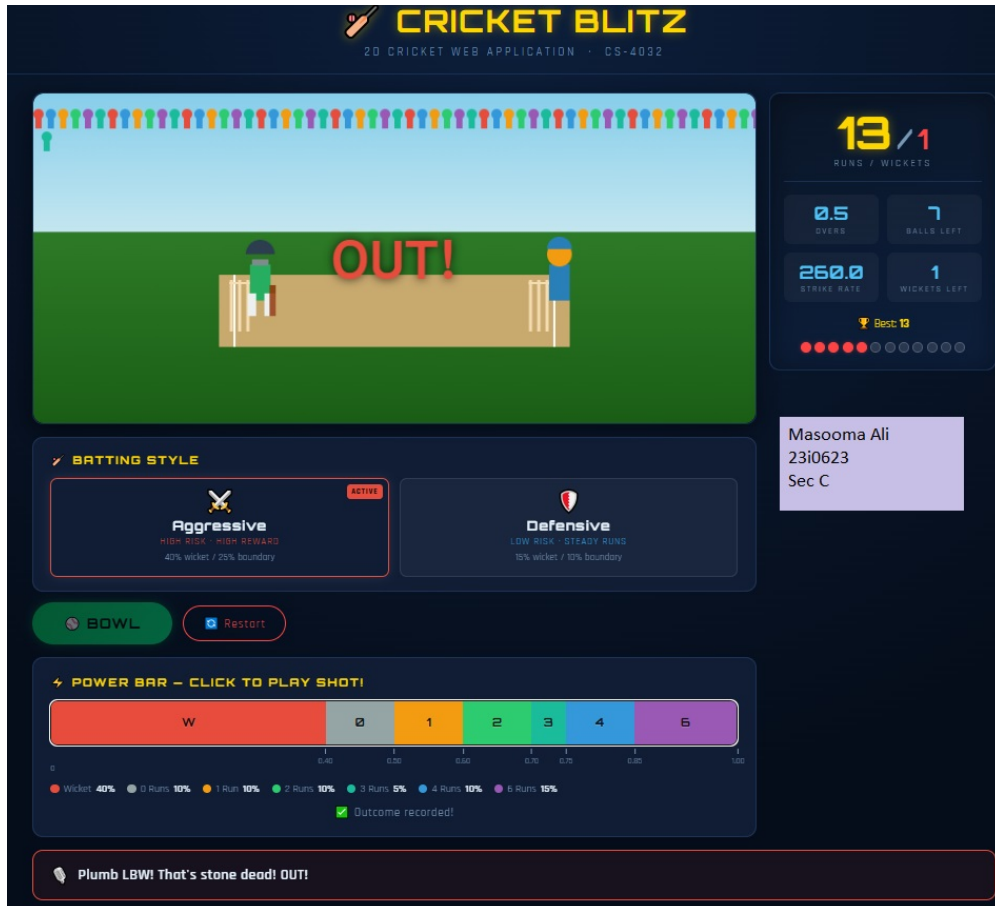


Figure 1.4 — Scoreboard showing live updates: 13/1, SR 260.0, 7 balls left, 1 wicket left. OUT overlay visible on field. Commentary shown. Sticky note: Masooma Ali | 23i0623 | Sec C.

SECTION 1.5

Game Over Screen — Innings Complete

The innings complete screen appears after all 12 balls are bowled. The final score is 36/1, achieved in 2.0 overs (12 balls) with a Strike Rate of 300.0. The summary panel shows all key statistics: Session Best (36), Balls Faced (12/12), and Wickets Lost (1/2). A 'Play Again' button allows instant restart.



Figure 1.5 — Innings Complete screen. Final score 36/1 in 2.0 overs. SR: 300.0. Sticky note: Masooma Ali | 23i0623 | Sec C.

SECTION 1.6

Game Over Screen — All Out

This screenshot shows the 'ALL OUT!' game-over condition, triggered when both wickets are lost before the 12 balls are completed. The match ended at 0/2 in just 1.2 overs (2 balls), with a Strike Rate of 0.0. Balls Faced is 2/12 and Wickets Lost is 2/2. This demonstrates the game correctly ending when the wicket limit is reached.

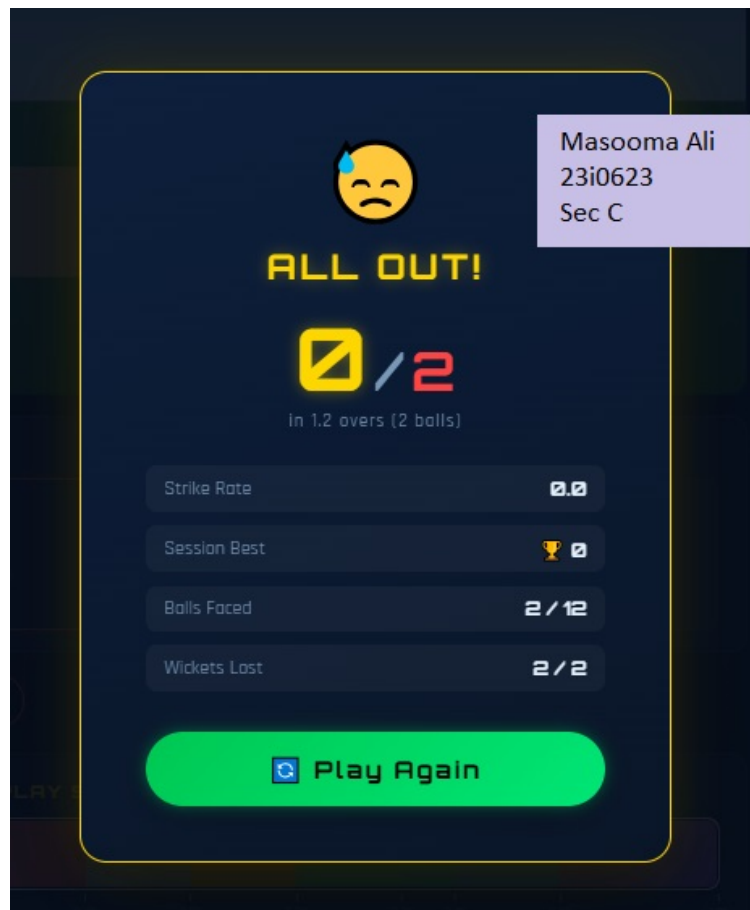


Figure 1.6 — ALL OUT game-over screen. Score 0/2 in 1.2 overs. Both wickets lost. Sticky note: Masooma Ali | 23i0623 | Sec C.

PART 2 — TECHNICAL EXPLANATION

SECTION 2.1

Game Logic and Working Mechanism

The application is structured around a **finite state machine** with five distinct phases managed by the custom React hook `useGameState.js`:

Phase	Description
IDLE	Waiting for the player to select a batting style and click BOWL.
BOWLING	Ball travel animation plays from bowler end to batsman end.
BATTING	Power bar slider is activated. Player must click to play the shot.
RESULT	Outcome is displayed for 1.8 seconds with commentary and animation.
GAME_OVER	Match has ended (all balls bowled or all wickets fallen). Summary shown.

Phase Transition Flow:

IDLE → BOWLING → BATTING → RESULT → IDLE (loops until game ends) → GAME_OVER

Delivery Sequence (step by step):

1. Player selects Aggressive or Defensive batting style.
2. Player clicks the BOWL button — phase transitions to BOWLING.
3. A `requestAnimationFrame` loop animates the ball from the bowler's coordinates to the batsman's coordinates over 900ms.
4. On animation completion, `onBowlingComplete()` is called — phase moves to BATTING.
5. The power bar slider activates and begins animating (ping-pong, left to right).
6. Player clicks anywhere on the power bar — the slider's current position (a float 0.0 to 1.0) is captured.
7. `resolveOutcome(sliderPosition, battingStyle)` compares the position against cumulative probability segments to return an outcome.
8. Runs are added, wickets incremented if out, ball count advances by 1.
9. Phase moves to RESULT — outcome text, commentary, and batting animation display for 1.8 seconds.
10. `useEffect` auto-calls `acknowledgeResult()` after 1.8s — phase resets to IDLE for next ball.
11. If `ballsBowled >= 12` or `wickets >= 2`, phase becomes GAME_OVER.

State Management:

All game state (`runs`, `wickets`, `ballsBowled`, `phase`, `battingStyle`, `commentary`, `lastOutcome`, `bestScore`) is managed inside a single custom hook — `useGameState.js` — using React's `useState`. State is never mutated directly; all updates use the functional form `setState(prev => ({...prev, ...changes}))` to ensure safe, predictable updates. Derived values (`strikeRate`, `oversDisplay`, `ballsRemaining`) are computed on render from state, so no redundant state is stored.

Match Rules Summary:

Rule	Value
Total Overs	2 overs (12 balls)
Total Wickets	2 wickets
Balls per Over	6 balls
Batting Styles	Aggressive (high risk) / Defensive (low risk)
Outcome Determination	Strictly by slider position — NO randomness

SECTION 2.2

Mapping of Probabilities to the Power Bar

Each batting style has **7 outcomes**, each assigned a probability value. The probabilities for each style sum exactly to **1.00**. The utility function `buildPowerBarSegments()` in `gameLogic.js` converts these raw probabilities into cumulative start/end ranges, which are then rendered as proportional width segments on the power bar.

Segment Width Formula:

```
Segment width (%) = probability x 100%
```

For example, a Wicket probability of 0.40 becomes a segment that occupies exactly 40% of the total bar width. This is implemented by setting the CSS width of each segment div to `probability * 100%`, so the visual representation is always mathematically accurate.

Outcome Resolution Formula:

```
if (sliderPosition >= segmentStart AND sliderPosition < segmentEnd) => outcome =  
segment.outcome
```

Aggressive Batting — Probability Distribution:

Outcome	Probability	Bar Width	Bar Range
Wicket	0.40	40%	0.00 — 0.40
0 Runs	0.10	10%	0.40 — 0.50
1 Run	0.10	10%	0.50 — 0.60
2 Runs	0.10	10%	0.60 — 0.70
3 Runs	0.05	5%	0.70 — 0.75
4 Runs	0.10	10%	0.75 — 0.85
6 Runs	0.15	15%	0.85 — 1.00
TOTAL	1.00	100%	

Defensive Batting — Probability Distribution:

Outcome	Probability	Bar Width	Bar Range
Wicket	0.15	15%	0.00 — 0.15
0 Runs	0.30	30%	0.15 — 0.45
1 Run	0.25	25%	0.45 — 0.70
2 Runs	0.15	15%	0.70 — 0.85
3 Runs	0.05	5%	0.85 — 0.90
4 Runs	0.07	7%	0.90 — 0.97
6 Runs	0.03	3%	0.97 — 1.00
TOTAL	1.00	100%	

Key Design Differences (Aggressive vs Defensive):

Aggressive style carries a 40% chance of a wicket and 25% chance of a boundary (4 or 6), making it high risk and high reward. Defensive style carries only 15% wicket probability and just 10% boundary chance,

but has a 55% probability of scoring 0 or 1 run safely. This directly affects gameplay strategy — aggressive is better when quick runs are needed, while defensive is safer when preserving wickets is the priority.

SECTION 2.3

Implementation of Animations

All animations are implemented using the **requestAnimationFrame (rAF)** API for smooth, frame-rate-synchronized rendering. The canvas is redrawn every frame inside a persistent render loop, and animation-specific state is stored in useRef objects to avoid triggering React re-renders on every frame.

A. Bowling Animation

Triggered when the game phase enters BOWLING via a useEffect hook watching the phase value. The ball's starting coordinates are set at the bowler's end of the pitch (x=490). A rAF loop runs over a fixed 900ms duration. On each frame:

- A progress variable t is calculated as: $t = (\text{currentTime} - \text{startTime}) / 900$
- t is passed through an ease-in-out quadratic function for smooth deceleration: $\text{eased} = t < 0.5 ? 2t^2 : -1 + (4 - 2t) * t$
- Ball X is interpolated from start to target using: $\text{ballX} = \text{startX} + (\text{targetX} - \text{startX}) * \text{eased}$
- A parabolic arc is added to Y: $\text{arcOffset} = -30 * \sin(\text{PI} * t)$, simulating a delivery bounce
- When t reaches 1.0, the animation stops, ball becomes invisible, and `onBowlingComplete()` advances the phase to BATTING

B. Batting Swing Animation

Triggered when the phase enters RESULT. A `batSwinging` flag and `batSwingDir` direction value are set on the stateRef object. Inside the persistent canvas render loop (which runs every frame regardless of phase):

- When `batSwinging` is true, `batAngle` increments by 8 degrees per frame in the forward direction
- When `batAngle` reaches 80 degrees (full swing), `batSwingDir` flips to -1 (reverse)
- When `batAngle` returns below -10 degrees, `batSwinging` is set to false and the bat resets to 0 degrees
- The bat is drawn on the canvas as a rotated rectangle using `ctx.rotate(batAngle * PI / 180)`, giving a realistic swing-and-follow-through effect

C. Boundary Particle Explosion (4s and 6s)

When the outcome is a FOUR or SIX, 30 particle objects are spawned at the batsman's position. Each particle has randomised properties:

- Position: batsman X/Y with small random offset
- Velocity: random v_x (-4 to +4) and v_y (-12 to -4) for upward explosion
- Size: radius between 3 and 8 pixels
- Colour: randomly chosen from 5 bright colours (gold, red, blue, green, purple)
- Alpha: starts at 1.0, decreases by 0.02 per frame

Each frame the main render loop updates particle positions ($x += v_x$, $y += v_y$), applies gravity ($v_y += 0.2$), and reduces alpha. Particles with $\alpha \leq 0$ are removed from the array. `ctx.globalAlpha` is set per particle before drawing.

D. Power Bar Slider Animation

Managed entirely by the custom hook **usePowerBar.js** using `requestAnimationFrame`. The slider animates in a ping-pong pattern:

- A `posRef` (`useRef`) stores the current position (0.0 to 1.0) to avoid stale closure issues
- A `dirRef` stores the current direction (+1 = left to right, -1 = right to left)
- Each frame: $\text{speed} = 1 / (\text{SLIDER_SPEED_MS} / 1000) = 0.5 \text{ units/second}$
- `posRef.current += dirRef.current * speed * deltaTime`
- When position hits 1.0 or 0.0, direction flips (ping-pong bounce)
- `setSliderPos(posRef.current)` triggers a React re-render each frame to move the DOM slider element
- When the player clicks, `stopSlider()` calls `cancelAnimationFrame()` to freeze the position
- The frozen `sliderPos` value is then passed directly to `playShot()` for outcome resolution

E. Result Overlay Animation

When the phase is `RESULT`, the canvas draw function renders large coloured outcome text (e.g. 'FOUR!', 'OUT!', 'SIX!') centred on the field using `ctx.fillText` with a `ctx.shadowBlur` of 10 for a glow effect. Simultaneously, the `Commentary` component uses a CSS transition (opacity 0 to 1, `translateY` 6px to 0) over 350ms to animate in the commentary text below the game.